4.12 Fundamentals of functional programming

4.12.1 Functional programming paradigm

4.12.1.1 Function type

Content	Additional information
Know that a function, f, has a function type f: A → B (where the type is A → B, A is the argument type, and B is the result type). Know that A is called the domain and B is called the co-domain. Know that the domain and co-domain are always subsets of objects in some data type.	Loosely speaking, a *function* is a rule that, for each element in some set A of inputs, assigns an output chosen from set B, but without necessarily using every member of B. For example, f: {a,b,c,…z} → {0,1,2,…,25} could use the rule that maps *a* to 0, *b* to 1, and so on, using all values which are members of set B. The *domain* is a set from which the function's input values are chosen. The *co-domain* is a set from which the function's output values are chosen. Not all of the co-domain's members need to be outputs.

4.12.1.2 First-class object

Content	Additional information
Know that a function is a first-class object in functional programming languages and in imperative programming languages that support such objects. This means that it can be an argument to another function as well as the result of a function call.	First-class objects (or values) are objects which may: • appear in expressions • be assigned to a variable • be assigned as arguments • be returned in function calls. For example, integers, floating-point values, characters and strings are first class objects in many programming languages.

4.12.1.3 Function application

Content	Additional information
Know that function application means a function applied to its arguments.	The process of giving particular inputs to a function is called *function application*, for example add(3,4) represents the application of the function *add* to integer arguments 3 and 4. The type of the function is *f: integer* x *integer* → *integer* where *integer* x *integer* is the Cartesian product of the set *integer* with itself. Although we would say that function *f* takes two arguments, in fact it takes only one argument, which is a pair, for example (3,4).

4.12.1.4 Partial function application

Content	Additional information
Know what is meant by partial function application for one, two and three argument functions and be able to use the notations shown opposite.	The function *add* takes two *integers* as arguments and gives an *integer* as a result. Viewed as follows in the partial function application scheme: *add: integer → (integer → integer)* add 4 returns a function which when applied to another integer adds 4 to that integer. The brackets may be dropped so function *add* becomes add: *integer → integer → integer* The function add is now viewed as taking one argument after another and returning a result of data type *integer*.

4.12.1.5 Composition of functions

Content	Additional information
Know what is meant by composition of functions.	The operation *functional composition* combines two functions to get a new function. Given two functions $f: A \rightarrow B$ $g: B \rightarrow C$ function $g \circ f$, called the *composition* of *g* and *f*, is a function whose domain is A and co-domain is C. If the domain and co-domains of *f* and *g* are $\mathbb{R}$, and $f(x) = (x + 2)$ and $g(y) = y^3$. Then $g \circ f = (x + 2)^3$ *f* is applied first and then *g* is applied to the result returned by *f*.